

Number

数位 DP

本页面将简要介绍数位 DP。

引入

数位是指把一个数字按照个、十、百、千等等一位一位地拆开，关注它每一位上的数字。如果拆的是十进制数，那么每一位数字都是 0~9，其他进制可类比十进制。

数位 DP：用来解决一类特定问题，这种问题比较好辨认，一般具有这几个特征：

1. 要求统计满足一定条件的数的数量（即，最终目的为计数）；
2. 这些条件经过转化后可以使用「数位」的思想去理解和判断；
3. 输入会提供一个数字区间（有时也只提供上界）来作为统计的限制；
4. 上界很大（比如），暴力枚举验证会超时。

数位 DP 的基本原理：

考虑人类计数的方式，最朴素的计数就是从小到大开始依次加一。但我们发现对于位数比较多的数，这样的过程中有许多重复的部分。例如，从 7000 数到 7999、从 8000 数到 8999、和从 9000 数到 9999 的过程非常相似，它们都是后三位从 000 变到 999，不一样的地方只有千位这一位，所以我们可以把这些过程归并起来，将这些过程中产生的计数答案也都存在一个通用的数组里。此数组根据题目具体要求设置状态，用递推或 DP 的方式进行状态转移。

数位 DP 中通常会利用常规计数问题技巧，比如把一个区间内的答案拆成两部分相减（即）

那么有了通用答案数组，接下来就是统计答案。统计答案可以选择记忆化搜索，也可以选择循环迭代递推。为了不重不漏地统计所有不超过上限的答案，要从高到低枚举每一位，再考虑每一位都可以填哪些数字，最后利用通用答案数组统计答案。

接下来我们具体看几道题目。

例题一

▼ 例 1 [Luogu P2602 数字计数](#)

题目大意：给定两个正整数，求在 中的所有整数中，每个数码（digit）各出现了多少次。

方法一

解释

发现对于满 位的数，所有数字出现的次数都是相同的，故设数组 为满 位的数中每个数字出现的次数，此时暂时不处理前导零。则有，这两部分前一个是来自前 位数字的贡献，后一个是来自第 位的数字的贡献。

有了 数组，我们来考虑如何统计答案。将上界按位分开，从高到低枚举，不贴着上界时，后面可以随便取值。贴着上界时，后面就只能取 到上界，分两部分分别计算贡献。最后考虑下前导零，第 位为前导 时，此时 到 位也都是，也就是多算了将 位填满的答案，需要额外减去。

实现

▼ 参考代码

```
1 #include <stdio.h>
2 using namespace std;
3 constexpr int N = 15;
4 using ll = long long;
5 ll l, r, dp[N], mi[N];
6 ll ans1[N], ans2[N];
7 int a[N];
8
9 void solve(ll n, ll *ans) {
10     ll tmp = n;
11     int len = 0;
12     while (n) a[++len] = n % 10, n /= 10;
13     for (int i = len; i >= 1; --i) {
14         for (int j = 0; j < 10; j++) ans[j] += dp[i - 1] * a[i];
15         for (int j = 0; j < a[i]; j++) ans[j] += mi[i - 1];
16         tmp -= mi[i - 1] * a[i], ans[a[i]] += tmp + 1;
17         ans[0] -= mi[i - 1];
18     }
19 }
20
21 int main() {
22     scanf("%lld%lld", &l, &r);
23     mi[0] = 1;
24     for (int i = 1; i <= 13; ++i) {
25         dp[i] = dp[i - 1] * 10 + mi[i - 1];
26         mi[i] = 10 * mi[i - 1];
27     }
28     solve(r, ans1), solve(l - 1, ans2);
29     for (int i = 0; i < 10; ++i) printf("%lld ", ans1[i] - ans2[i]);
30     return 0;
31 }
```

方法二

解释

此题也可以使用记忆化搜索。表示不贴上限、无前导零时，位数为 的答案。

详见代码注释

过程

▼ 参考代码

```

1 #include <stdio>
2 #include <string>
3 #include <iostream>
4 using namespace std;
5 using ll = long long;
6 constexpr int N = 50005;
7 ll a, b;
8 ll f[15], ksm[15], p[15], now[15];
9
10 ll dfs(int u, int x, bool f0,
11        bool lim) { // u 表示位数, f0 是否有前导零, lim 是否都贴在上限上
12     if (!u) {
13         if (f0) f0 = false;
14         return 0;
15     }
16     if (!lim && !f0 && (~f[u])) return f[u];
17     ll cnt = 0;
18     int lst = lim ? p[u] : 9;
19     for (int i = 0; i <= lst; i++) { // 枚举这位要填的数字
20         if (f0 && i == 0)
21             cnt += dfs(u - 1, x, 1, lim && i == lst); // 处理前导零
22         else if (i == x && lim && i == lst)
23             cnt += now[u - 1] + 1 +
24                 dfs(u - 1, x, 0,
25                    lim && i == lst); // 此时枚举的前几位都贴在给定的上限上。
26         else if (i == x)
27             cnt += ksm[u - 1] + dfs(u - 1, x, 0, lim && i == lst);
28         else
29             cnt += dfs(u - 1, x, 0, lim && i == lst);
30     }
31     if ((!lim) && (!f0)) f[u] = cnt; // 只有不贴着上限和没有前导零才能记忆
32     return cnt;
33 }
34
35 ll gans(ll d, int dig) {
36     int len = 0;
37     memset(f, -1, sizeof(f));
38     while (d) {
39         p[++len] = d % 10;
40         d /= 10;
41         now[len] = now[len - 1] + p[len] * ksm[len - 1];
42     }
43     return dfs(len, dig, 1, 1);
44 }
45
46 int main() {
47     scanf("%lld%lld", &a, &b);
48     ksm[0] = 1;
49     for (int i = 1; i <= 12; i++) ksm[i] = ksm[i - 1] * 1011;
50     for (int i = 0; i < 9; i++) printf("%lld ", gans(b, i) - gans(a - 1, i));
51     printf("%lld\n", gans(b, 9) - gans(a - 1, 9));
52     return 0;
53 }

```

例题二

▼ 例 2 [HDU 2089 不要 62](#)

题面大意：统计一个区间内数位上不能有 4 也不能有连续的 62 的数有多少。

解释

没有 4 的话在枚举的时候判断一下，不枚举 4 就可以保证状态合法了，所以这个约束没有记忆化的必要，而对于 62 的话，涉及到两位，当前一位是 6 或者不是 6 这两种不同情况计数是不相同的，所以要用状态来记录不同的方案数。表示当前第 i 位，前一位是否是 6 的状态，这里只需要取 0 和 1 两种状态就可以了，不是 6 的情况可视为同种，不会影响计数。

实现

▼ 参考代码

```
1 #include <LTCstdio>
2 #include <LTCstring>
3 #include <LTiostream>
4 using namespace std;
5 int x, y, dp[15][3], p[50];
6
7 void pre() {
8     memset(dp, 0, sizeof(dp));
9     dp[0][0] = 1;
10    for (int i = 1; i <= 10; i++) {
11        dp[i][0] = dp[i - 1][0] * 9 - dp[i - 1][1];
12        dp[i][1] = dp[i - 1][0];
13        dp[i][2] = dp[i - 1][2] * 10 + dp[i - 1][1] + dp[i - 1][0];
14    }
15 }
16
17 int cal(int x) {
18     int cnt = 0, ans = 0, tmp = x;
19     while (x) {
20         p[++cnt] = x % 10;
21         x /= 10;
22     }
23     bool flag = false;
24     p[cnt + 1] = 0;
25     for (int i = cnt; i; i--) { // 从高到低枚举数位
26         ans += p[i] * dp[i - 1][2];
27         if (flag)
28             ans += p[i] * dp[i - 1][0];
29         else {
30             if (p[i] > 4) ans += dp[i - 1][0];
31             if (p[i] > 6) ans += dp[i - 1][1];
32             if (p[i] > 2 && p[i + 1] == 6) ans += dp[i][1];
33             if (p[i] == 4 || (p[i] == 2 && p[i + 1] == 6)) flag = true;
34         }
35     }
36     return tmp - ans;
37 }
38
39 int main() {
40     pre();
41     while (~scanf("%d%d", &x, &y)) {
42         if (!x && !y) break;
43         x = min(x, y), y = max(x, y);
44         printf("%d\n", cal(y + 1) - cal(x));
45     }
46     return 0;
47 }
```

例题三

▼ 例 3 [SCOI2009 windy 数](#)

题目大意：给定一个区间，求其中满足条件不含前导 0 且相邻两个数字相差至少为 2 的数字个数。

解释

首先我们将问题转化成更加简单的形式。设 $f(x, st, op)$ 表示在区间 $[x, st]$ 中满足条件的数的数量，那么所求的答案就是 $f(1, 11, 1)$ 。

对于一个小于 st 的数，它从高到低肯定出现某一位，使得这一位上的数值小于 st 这一位上对应的数值。而之前的所有位都和 st 上的位相等。

有了这个性质，我们可以定义 $f(x, st, op)$ 表示当前将要考虑的是从高到低的第 st 位，当前该前缀的状态为 op 且前缀和当前求解的数字的大小关系是（ $op=1$ 表示等于， $op=0$ 表示小于）时的数字个数。在本题中，这个前缀的状态就是上一位的值，因为当前将要确定的位不能取哪些数只和上一位有关。在其他题目中，这个值可以是：前缀的数字和，前缀所有数字的，该前缀取模某个数的余数，也有两种或多种合用的情况。

写出 **状态转移方程**：

这里的 st 就是当前枚举的下一位的值，而 op 就是当前能取到的最高位。因为如果 $op=1$ ，那么你在这一位上取的值一定不能大于求解的数字上该位的值，否则则没有限制。

我们发现，尽管前缀所选择的状态不同，而 $f(x, st, op)$ 的三个参数相同，答案就是一样的。为了防止这个答案被计算多次，可以使用 [记忆化搜索](#) 的方式实现。

实现

▼ 参考代码

```
1 int dfs(int x, int st, int op) // op=1 =; op=0 <
2 {
3     if (!x) return 1;
4     if (!op && ~f[x][st]) return f[x][st];
5     int maxx = op ? dim[x] : 9, ret = 0;
6     for (int i = 0; i <= maxx; i++) {
7         if (abs(st - i) < 2) continue;
8         if (st == 11 && i == 0)
9             ret += dfs(x - 1, 11, op & (i == maxx));
10        else
11            ret += dfs(x - 1, i, op & (i == maxx));
12    }
13    if (!op) f[x][st] = ret;
14    return ret;
15 }
16
17 int solve(int x) {
18     memset(f, -1, sizeof f);
19     dim.clear();
20     dim.push_back(-1);
21     int t = x;
22     while (x) {
23         dim.push_back(x % 10);
24         x /= 10;
25     }
26     return dfs(dim.size() - 1, 11, 1);
27 }
```

例题四

▼ 例 4. [SPOJMYQ10](#)

题面大意：假如手写下 n 之间所有整数，会有多少数看起来和在镜子里看起来一模一样？（）

解释

注：由于这里考虑到的镜像，只有 的镜像是自己本身。所以，这里的「一模一样」并不是传统意义上的回文串，而是只含有 的回文串。

首先，在数位 DP 过程中，显然只有 能被选中。

其次，由于数值超过 long long 范围，所以 不再适用（高精度比较繁琐），而是需要对 是否合法进行判断，得出：。

镜像解决了，如何判断回文？

我们需要用一个小数组记录一下之前的值。在未超过一半的长度时，只要不超上限就行；在超过一半的长度时，还需要判断是否和与之「镜面对称」的位相等。

需要额外注意的是，这道题的记忆化部分，不能用 `memset`，否则会导致超时。

实现

▼ 参考代码

```
1 int check(char cc[]) { // n 的特判
2     int strc = strlen(cc);
3     for (int i = 0; i < strc; ++i) {
4         if (!(cc[i] == cc[strc - i - 1] &&
5             (cc[i] == '1' || cc[i] == '8' || cc[i] == '0'))))
6             return 0ll;
7     }
8     return 1ll;
9 }
10
11 // now: 当前位, eff: 有效位, fulc: 是否全顶格, ful0: 是否全0
12 int dfs(int now, int eff, bool ful0, bool fulc) {
13     if (now == 0) return 1ll;
14     if (!fulc && f[now][eff][ful0] != -1) // 记忆化
15         return f[now][eff][ful0];
16
17     int res = 0, maxk = fulc ? dig[now] : 9;
18     for (int i = 0; i <= maxk; ++i) {
19         if (i != 0 && i != 1 && i != 8) continue;
20         b[now] = i;
21         if (ful0 && i == 0) // 全前导 0
22             res += dfs(now - 1, eff - 1, 1, 0);
23         else if (now > eff / 2) // 未过半程
24             res += dfs(now - 1, eff, 0, fulc && (dig[now] == i)); // 已过半程
25         else if (b[now] == b[eff - now + 1])
26             res += dfs(now - 1, eff, 0, fulc && (dig[now] == i));
27     }
28     if (!fulc) f[now][eff][ful0] = res;
29     return res;
30 }
31
32 char cc1[100], cc2[100];
33 int strc, ansm, ansn;
34
35 int get(char cc[]) { // 处理封装
36     strc = strlen(cc);
37     for (int i = 0; i < strc; ++i) dig[strc - i] = cc[i] - '0';
38     return dfs(strc, strc, 1, 1);
39 }
40
41 scanf("%s%s", cc1, cc2);
42 printf("%lld\n", get(cc2) - get(cc1) + check(cc1));
```

例题五

▼ 例 5.P3311 数数

题面：我们称一个正整数 是幸运数，当且仅当它的十进制表示中不包含数字串集合 中任意一个元素作为其子串。例如当 时， 是幸运数， 、 、 不是幸运数。给定 和 ， 计算不大于 的幸运数个数。答案对 取模。

， ， ， ， 其中 表示字符串 的长度。没有前导，但是 可能有前导。

解释

阅读题面发现，如果将数字看成字符串，那么这就是需要完成一个多模匹配，自然而然就想到 AC 自动机。普通数位 DP 中，先从高到低枚举数位，再枚举每一位都填什么，在这道题中，我们也就自然地转化为枚举已经填好的位数，再枚举此时停在 AC 自动机上的哪个节点，然后从当前节点转移到它在 AC 自动机上的子节点。

设 表示当前从高到低已经填了 位（即在 AC 自动机上走过了 条边），此时停在标号为 的节点上，当前是否正好贴着上界。

至于题目中的「不包含」条件，只需在 AC 自动机上给每个模式串的结尾节点都打上标记，DP 过程中一旦遇上这些结尾节点就跳过即可。

转移很好想，详见代码主函数部分。

实现

▼ 参考代码

```
1 #include <cstdio>
2 #include <cstring>
3 #include <queue>
4 using namespace std;
5 using ll = long long;
6 constexpr int N = 1505;
7 constexpr int mod = 1000000007;
8 int n, m;
9 char s[N], c[N];
10 int ch[N][10], fail[N], ed[N], tot, len;
11
12 void insert() {
13     int now = 0;
14     int L = strlen(s);
15     for (int i = 0; i < L; ++i) {
16         if (!ch[now][s[i] - '0']) ch[now][s[i] - '0'] = ++tot;
17         now = ch[now][s[i] - '0'];
18     }
19     ed[now] = 1;
20 }
21
22 queue<int> q;
23
24 void build() {
25     for (int i = 0; i < 10; ++i)
26         if (ch[0][i]) q.push(ch[0][i]);
27     while (!q.empty()) {
28         int u = q.front();
29         q.pop();
30         for (int i = 0; i < 10; ++i) {
31             if (ch[u][i]) {
32                 fail[ch[u][i]] = ch[fail[u]][i], q.push(ch[u][i]),
33                 ed[ch[u][i]] |= ed[fail[ch[u][i]]];
34             } else
35                 ch[u][i] = ch[fail[u]][i];
36         }
37     }
```

```

38  ch[0][0] = 0;
39 }
40
41 ll f[N][N][2], ans;
42
43 void add(ll &x, ll y) { x = (x + y) % mod; }
44
45 int main() {
46     scanf("%s", c);
47     n = strlen(c);
48     scanf("%d", &m);
49     for (int i = 1; i <= m; ++i) scanf("%s", s), insert();
50     build();
51     f[0][0][1] = 1;
52     for (int i = 0; i < n; ++i) {
53         for (int j = 0; j <= tot; ++j) {
54             if (ed[j]) continue;
55             for (int k = 0; k < 10; ++k) {
56                 if (ed[ch[j][k]]) continue;
57                 add(f[i + 1][ch[j][k]][0], f[i][j][0]);
58                 if (k < c[i] - '0') add(f[i + 1][ch[j][k]][0], f[i][j][1]);
59                 if (k == c[i] - '0') add(f[i + 1][ch[j][k]][1], f[i][j][1]);
60             }
61         }
62     }
63     for (int j = 0; j <= tot; ++j) {
64         if (ed[j]) continue;
65         add(ans, f[n][j][0]);
66         add(ans, f[n][j][1]);
67     }
68     printf("%lld\n", ans - 1);
69     return 0;
70 }

```

此题可以很好地帮助理解数位 DP 的原理。

习题

[Ahoi2009 self 同类分布](#)

[洛谷 P3413 SAC#1 - 萌数](#)

[HDU 6148 Valley Number](#)

[CF55D Beautiful numbers](#)

[CF628D Magic Numbers](#)

本页面最近更新：2025/4/13 12:34:35, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者： [Alphnia](#), [billchenchina](#), [ChungZH](#), [Early0v0](#), [Enter-tainer](#), [GavinZhengOI](#), [greyqz](#), [H-J-Granger](#), [Henry-ZHR](#), [hsfzLZH1](#), [iamtwz](#), [lr1d](#), [isdanni](#), [ksyx](#), [ouuan](#), [Promise2679](#), [SamZhangQingChuan](#), [sshwy](#), [StudyingFather](#), [thredreams](#), [Tiphereth-A](#), [Xeonacid](#)

本页面的全部内容 [在 CC BY-SA 4.0 和 SATA 协议之条款下](#) 提供，附加条款亦可能应用